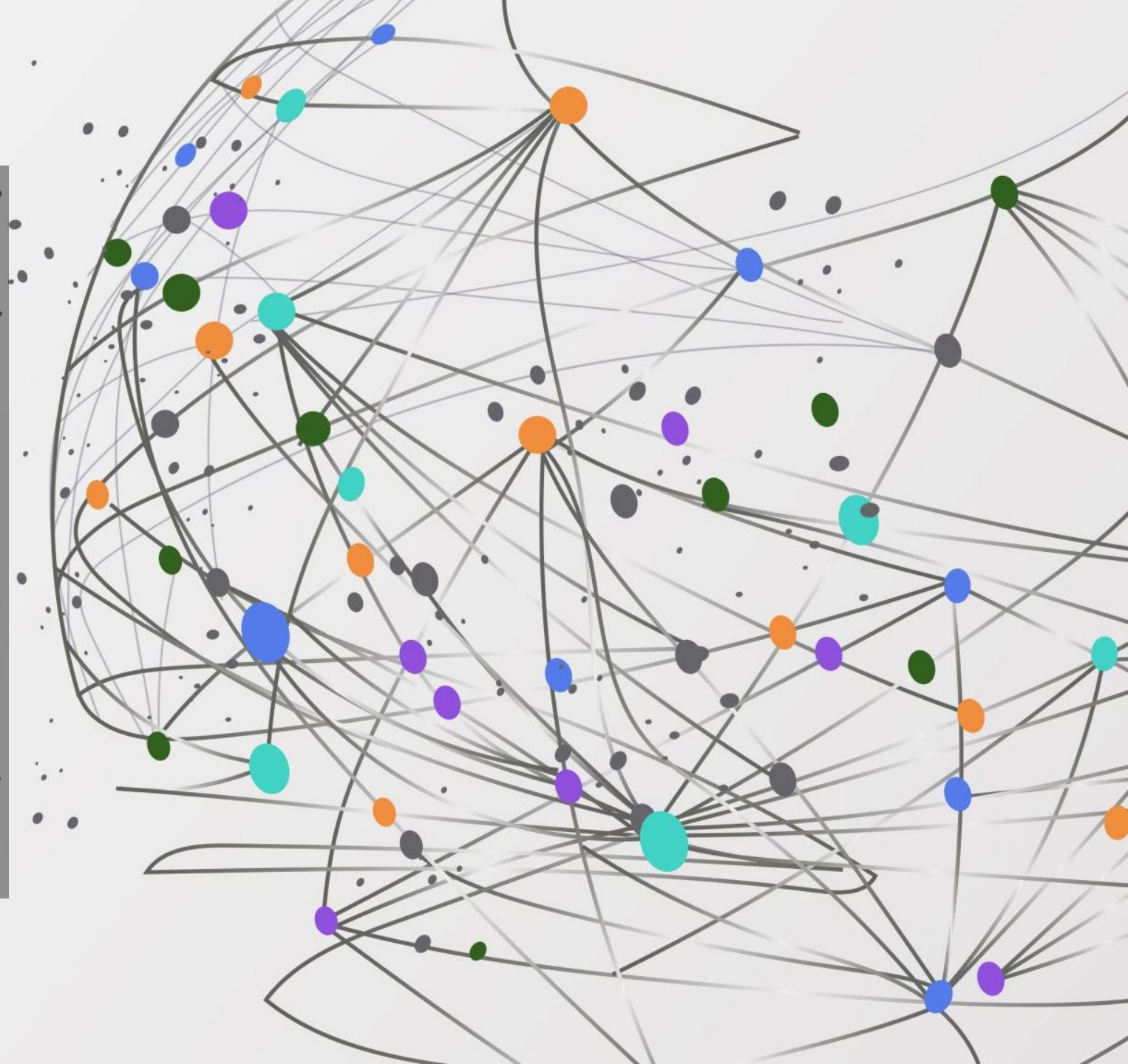
Machine Learning Project

BY BEN, BRIAN, ERIC, KAI



Project Overview

Goal: recognize spoken commands (yes, no, stop, go) using machine learning

Audio signals are converted into mel spectrogram images

A Convolutional Neural Network (CNN) is trained to classify commands

Implemented in MATLAB using the Google Speech Commands dataset

Dataset Description

- Used the Google Speech Commands Dataset v0.02 containing short spoken word recordings
- Each clip is 1 second long with a 16 kHz sampling rate
- Commands used in this stage of the project: yes, no, stop, go
- Dataset was split into training (80%) and testing (20%)

Importing the Dataset into MATLAB

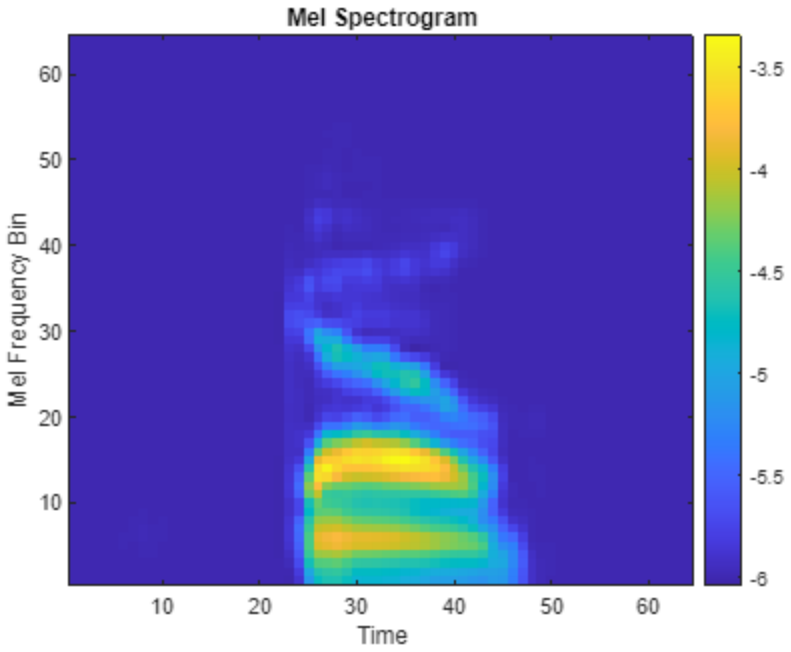
- Audio files were imported using MATLAB's **`audioDatastore`** function
- The datastore automatically loads **`.wav`** files from all dataset subfolders
- Folder names are used as **class labels** for each command
- This allows easy iteration through the dataset for preprocessing and training

```
datasetPath = "C:\UAlbany\2026\Spring 2026\ECE 465 - Machine Learning\speech_commands_v0.02";

ads = audioDatastore(datasetPath,'IncludeSubfolders', true, 'FileExtensions','.wav', 'LabelSource','foldernames');
```

Feature Extraction

```
[audio, fs] = audioread(adsTrain.Files{i});

melSpec = melSpectrogram(audio, fs);
melSpec = log10(melSpec + 1e-6);

melSpec = imresize(melSpec, [64 64]);
```

- Raw audio signals are converted into **mel spectrograms**
- Mel spectrograms represent **frequency content over time**
- Log scaling is applied to emphasize important speech features
- Each spectrogram is resized to **64 × 64** to serve as CNN input



Preprocessing Steps

- Each `.wav` file is converted into a **mel spectrogram image**
- Spectrograms are resized to **64 × 64 grayscale images**
- Images are stored in arrays **XTrain/XTest** with labels **YTrain/YTest**
- This creates a dataset suitable for **CNN training**

```
%Split into Training and Testing data
commands = categorical(["yes","no","stop","go"]);
ads = subset(ads, ismember(ads.Labels, commands));

[adsTrain, adsTest] = splitEachLabel(ads, 0.8);

%get mel spectrograms for all training files
numTrainFiles = numel(adsTrain.Files);
XTrain = zeros(64,64,1,numTrainFiles);
YTrain = adsTrain.Labels;

for i = 1:numTrainFiles
    [audio, fs] = audioread(adsTrain.Files{i});

    melSpec = melSpectrogram(audio, fs);
    melSpec = log10(melSpec + 1e-6);

    melSpec = imresize(melSpec, [64 64]);

    XTrain(:,:,1,i) = melSpec;
end

%do same for test files
numTestFiles = numel(adsTest.Files);
XTest = zeros(64,64,1,numTestFiles);
YTest = adsTest.Labels;

for i = 1:numTestFiles
    [audio, fs] = audioread(adsTest.Files{i});

    melSpec = melSpectrogram(audio, fs);
    melSpec = log10(melSpec + 1e-6);

    melSpec = imresize(melSpec, [64 64]);

    XTest(:,:,1,i) = melSpec;
end
```

Model Architecture

- A **Convolutional Neural Network (CNN)** is used for classification
  - Convolution layers detect **patterns in spectrogram images**
- Pooling layers reduce dimensionality while preserving important features
- Final fully connected layer maps extracted features to the command classes

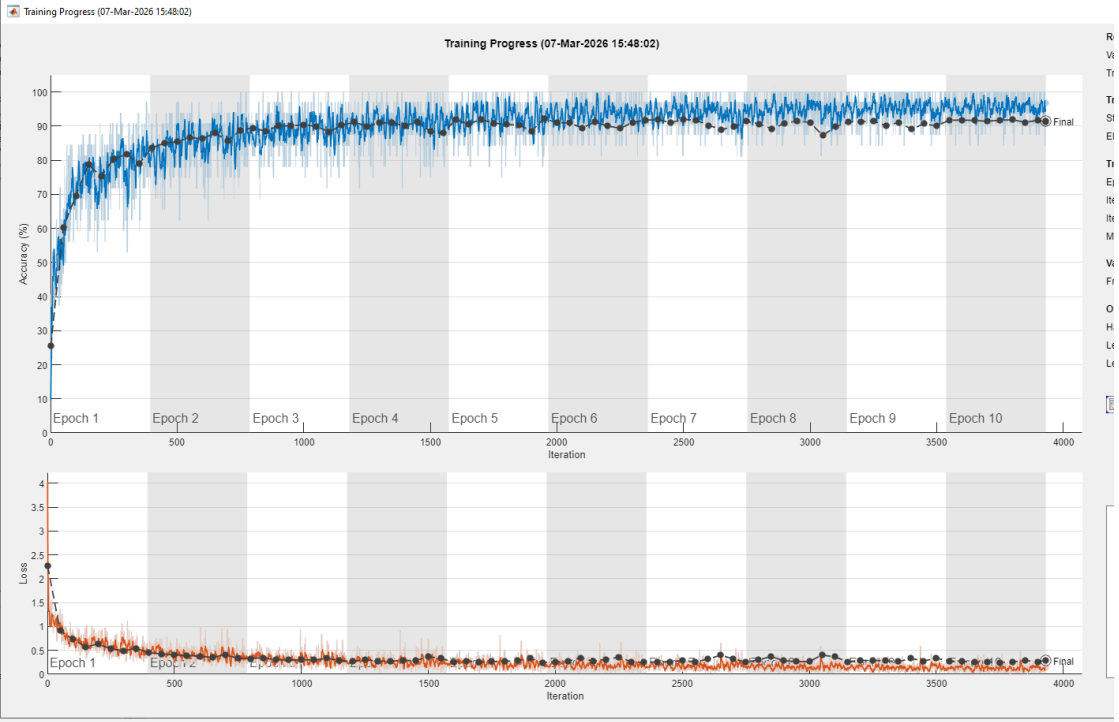
```
layers = [
    %tells MATLAB each input is a 64x64 grayscale image
    imageInputLayer([64 64 1])

    %applies 8 filters to detect simple patterns in the spectrogram
    convolution2dLayer(3,8,Padding="same")

    %adds nonlinearity so the network can learn more than just simple linear patterns
    reluLayer

    %reduces the image size and keeps the strongest features
    maxPooling2dLayer(2,Stride=2)

    %%This acts as a second layer of learning, now learning patterns
    convolution2dLayer(3,16,Padding="same")
    batchNormalizationLayer
    reluLayer

    maxPooling2dLayer(2,Stride=2)

    %maps the extracted features to your command classes
    fullyConnectedLayer(numel(categories(YTrain)))

    %turns outputs into class probabilities
    softmaxLayer

    %computes the classification loss during training
    classificationLayer
];
```

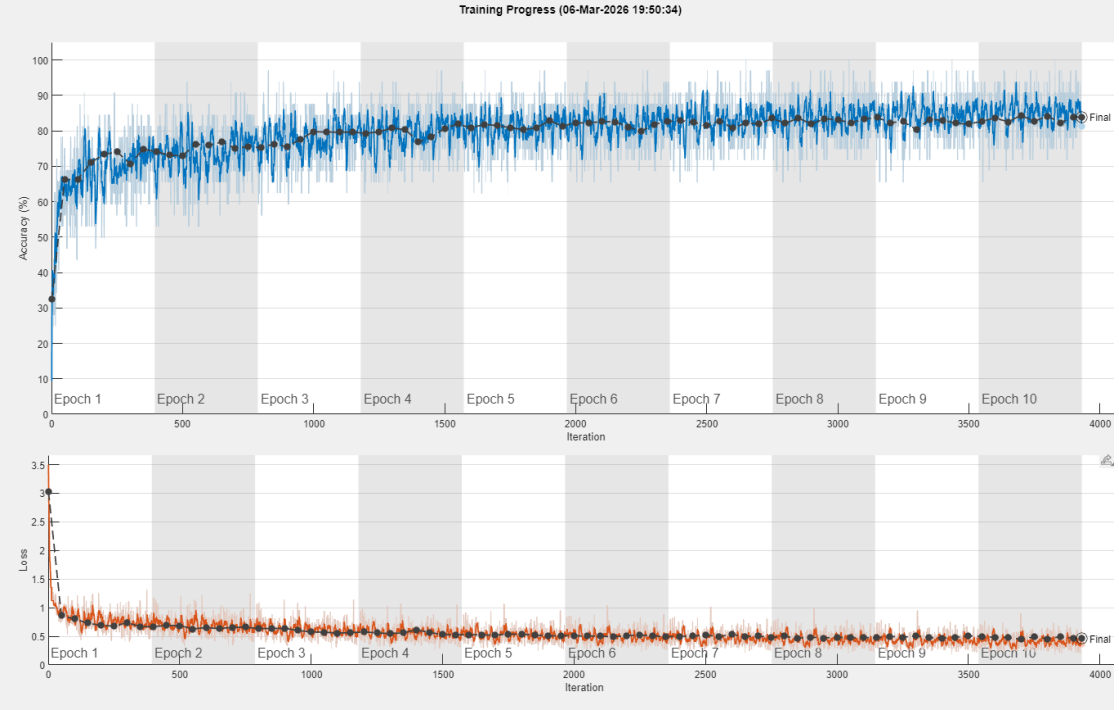
Training Configuration

```
options = trainingOptions("adam","MaxEpochs", 10,"MiniBatchSize", 32,"ValidationData", {XTest, YTest},"Verbose", false, "Plots", "training-progress");
```

Parameters given to training set:

- Optimizer: Adam
  - 'Adam' updates data's weight iteratively through each epoch
  - Adam was chosen due to its efficiency with audio datasets
- Epochs: 10
  - Number of complete tests ran through each batch.
    - This allows the model to better classify and run through the same data several times for more accurate results
- Batch size: 32
  - Each batch the program uses 32 files





Initial Results

- Accuracy
  - First training ~84%
  - Second training ~94%
    - After adding a second convolution layer accuracy increases by 10%
- Epoch
  - Each training runs through datasets 10 times
- Losses
  - Inaccurate classifications during training (lower number is better)

Live Demonstration

Conclusions:

- Developed a speech command recognition system using MATLAB and a Convolutional Neural Network (CNN).
- Converted audio recordings into Mel spectrograms, allowing the model to learn patterns in speech as image data.
- Trained the network to classify four commands: “yes,” “no,” “stop,” and “go.”
- Evaluated performance using a separate test dataset to measure classification accuracy.
- Implemented real-time prediction by recording a new audio sample and classifying it with the trained model.
- Key Takeaway: Transforming audio into spectrogram images enables CNNs to effectively learn and recognize spoken commands.

Future Work

Add	Add	Add	Add
A third convolution layer	Noise to the audio files to make model more robust	The ability to identify people	more commands (optional)

Questions?